



南開大學
Nankai University

数据挖掘实验报告

学 号:2112054

姓 名:王赫明

年 级:2021 级

学 院:统计与数据科学学院

专 业:统计学

完成日期: 2023 年 11 月 30 日

目录

1 第一次上机实验（垂直平分分类器）	4
1.1 实验要求	4
1.2 数据分析与处理	4
1.3 实验步骤与原理	4
1.4 实验结果与分析	4
1.5 实验代码	5
2 第二次上机实验（提取图像纹理特征）	7
2.1 实验要求	7
2.2 数据分析与处理	7
2.3 实验步骤与原理	7
2.4 实验结论与分析	8
2.5 实验代码	9
3 第三次上机实验（朴素 Bayes 预测）	11
3.1 实验要求	11
3.2 数据分析与处理	11
3.3 实验步骤与原理	11
3.4 实验结论与分析	12
3.5 实验代码	12
4 第四次上机实验（决策树预测）	14
4.1 实验要求	14
4.2 数据分析与处理	14
4.3 实验步骤与原理	14
4.4 实验结论与分析	15
4.5 实验代码	15
5 第五次上机实验（BP 神经网络）	17
5.1 实验要求	17
5.2 数据分析与处理	17

5.3 实验步骤与原理	17
5.4 实验结论与分析	18
5.5 实验代码	18

第一章 第一次上机实验（垂直平分分类器）

1.1 实验要求

- 用训练集训练一个垂直平分分类器，对测试集进行分类。（Python）

1.2 数据分析与处理

是一个 $C=2$, $D=2$ 的两类二维分类问题，训练集和测试集各含 399 条数据。

1.3 实验步骤与原理

· 实验原理

基于两类样本的中心 m_1, m_2 （均值点），作垂直平分线，以此为决策边界。记该直线方程为 $\omega^T x + \omega_0 = 0$ ，从而判别函数为 $g(x) = \omega^T x + \omega_0$ 。由样本的几何关系计算：

$$g(x) = (m_1 - m_2)^T \left(x - \frac{m_1 + m_2}{2} \right) \quad (1.1)$$

对于分类标签未知的待判别样本 x ，基于上述判别函数的判别准则如下：

$$\begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

· 实验步骤

- Step1: 导入数据，定义训练集与测试集；
- Step2: 计算均值向量，确定垂直平分线方程，定义判别函数；
- Step3: 计算测试集数据的响应，根据判别准则将其归类，计算分类准确率；
- Step4: 作相关图像，进行结果可视化输出。

1.4 实验结果与分析

测试集共 399 条数据，其中正确分类 396 条，错误分类 3 条，准确率为 99.25%。

决策面（垂直平分线）方程为：

$$6.254x + 5.879y - 0.238 = 0 \quad (1.2)$$

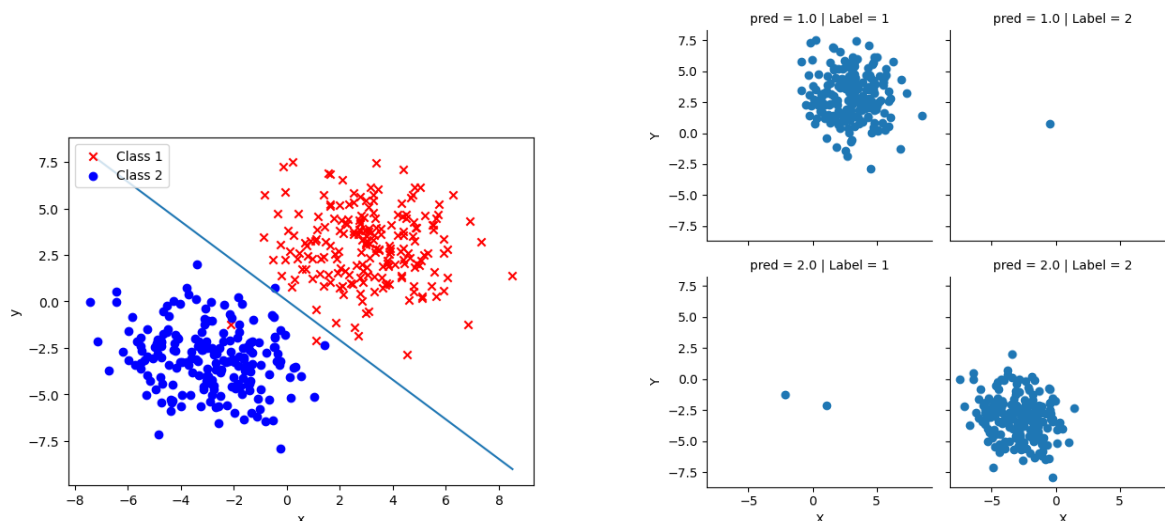


图 1.1: 实验 1 结果

1.5 实验代码

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib as mpl
5 import seaborn as sns
6
7 #导入数据
8 train = pd.read_csv('train_1.csv')
9 test = pd.read_csv('test_1.csv')
10 x1 = train[train['Label']==1]
11 x2 = train[train['Label']==2]
12 m1 = np.mean(x1).values[0:2]
13 m2 = np.mean(x2).values[0:2]
14
15 #定义判别函数
16 def g(x):
17     return np.dot((m1-m2).T, (x-0.5*(m1+m2)))
18
19 #对测试集数据进行分类
20 test_pred=np.zeros(400)
21 for i in range(400):
22     x = np.array(test.iloc[i,0:2])
23     if g(x)> 0:
24         test_pred[i] = 1
25     else:
26         test_pred[i] = 2

```

```
27 test['pred'] = test_pred
28 accuracy_num = 0
29
30 #计算分类准确率
31 for i in range(399):
32     if test['Label'][i] == test_list[i]:
33         accuracy_num +=1
34 accuracy_rate = accuracy_num/399
35 print("测试集上的分类准确率为: ",accuracy_rate)
36
37 #绘制测试集样本散点
38 x1_test=test[test['Label']==1]
39 x2_test=test[test['Label']==2]
40 plt.scatter(x1_test.iloc[:, 0], x1_test.iloc[:,
41 1], c = "red", marker='x',
42 label='Class 1')
43 plt.scatter(x2_test.iloc[:, 0], x2_test.iloc[:,
44 1], c = "blue", marker='o',
45 label='Class 2')
46
47 #绘制决策直面 (垂直平分线)
48 def f(x):
49     return -(-0.5*np.dot((m1-m2), (m1+m2)) + (m1-m2)[0]*x) / (m1-m2)[1]
50 x=[min(test['X']),max(test['X'])]
51 y=[f(x[0]),f(x[1])]
52 plt.plot(x,y)
53 plt.xlabel('x')
54 plt.ylabel('y')
55 plt.legend(loc=2)
56 plt.show()
57
58 #绘制FacetGrid图表
59 facet = sns.FacetGrid(test, col='Label',
60 row='pred')
61 facet.map(plt.scatter, 'X', 'Y')
62 plt.show()
```

第二章 第二次上机实验 (提取图像纹理特征)

2.1 实验要求

- 给定若干张图像，利用 GLCM 或者 LBP 方法对这些图像进行特征提取
- 将最终提取到的特征通过 plot 的形式展示，直观对比不同纹理提取到的特征
- 图象是 $W * H * 3$ 的矩阵，使用 Python 编程实现

注：本次实验采取 LBP 方法进行图片纹理特征提取与分析。

2.2 数据分析与处理

待处理图片是静态的 $W*H*3$ 矩阵 (RGB 通道构成的静态彩图)，分别为 10 张 cloud 图片与 10 张 forest 图。

由于原始图片是彩色的，在进行特征提取时要注意对其进行灰度转化。

2.3 实验步骤与原理

· 实验原理

· LBP 的基础定义

LBP 算子定义为在 3×3 的窗口内，以窗口中心像素为阈值，将相邻的 8 个像素的灰度值与其进行比较 (从左上角像素，顺时针旋转)，若周围像素值大于等于中心像素值，则该像素点的位置被标记为 1，否则为 0。由此可产生 8 位二进制数，将其进一步转换为十进制数即 LBP 编码 (256 种)。

example	thresholded	weights																											
<table border="1"> <tr><td>6</td><td>5</td><td>2</td></tr> <tr><td>7</td><td>6</td><td>1</td></tr> <tr><td>9</td><td>8</td><td>7</td></tr> </table>	6	5	2	7	6	1	9	8	7	<table border="1"> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td></td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	1	0	0	1		0	1	1	1	<table border="1"> <tr><td>1</td><td>2</td><td>4</td></tr> <tr><td>128</td><td></td><td>8</td></tr> <tr><td>64</td><td>32</td><td>16</td></tr> </table>	1	2	4	128		8	64	32	16
6	5	2																											
7	6	1																											
9	8	7																											
1	0	0																											
1		0																											
1	1	1																											
1	2	4																											
128		8																											
64	32	16																											
Pattern = 11110001																													
LBP = $1 + 16 + 32 + 64 + 128 = 241$																													

图 2.1: Simple example: Calculation of LBP

· LBP 有关计算式

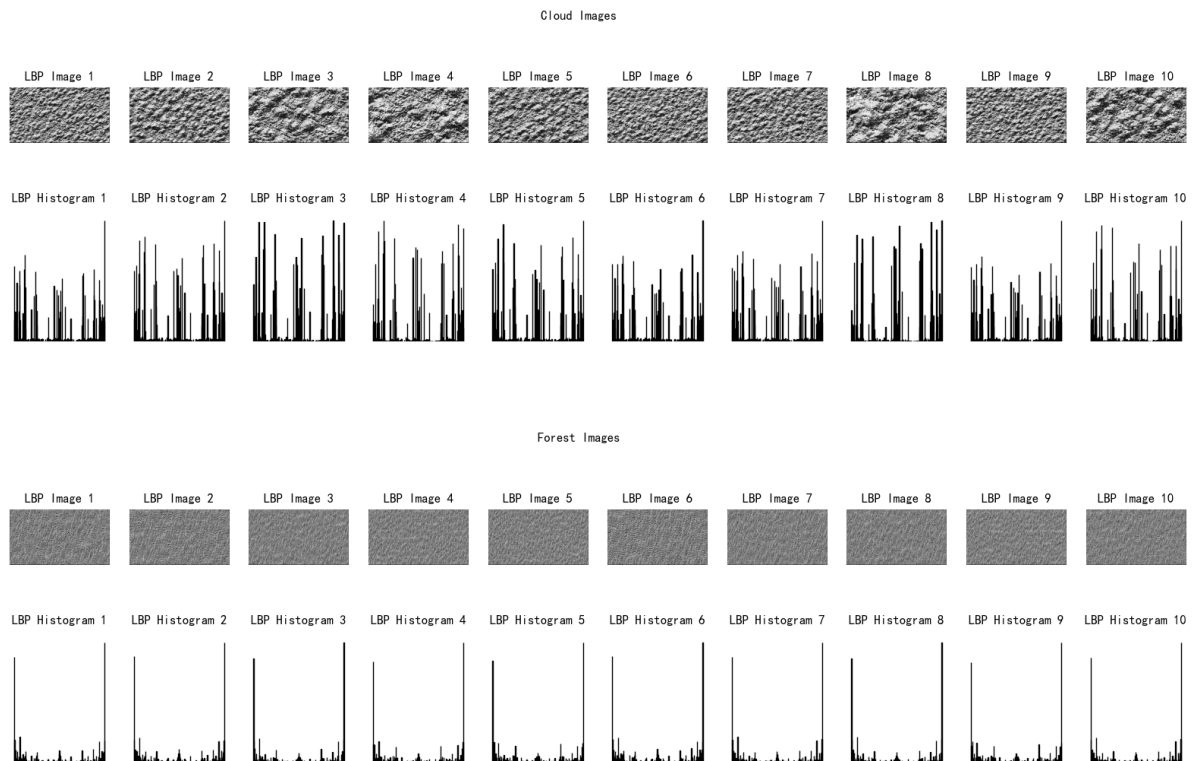
$$LBP(x_c, y_c) = \sum_{p=0}^{p-1} 2^p \text{sgn}(i_p - i_c) \quad (2.1)$$

x_c, y_c 是中心像素的坐标, p 为邻域的第 p 个像素 (以 8-邻域为例子, p 取值为 0-7), i_c 是该方形区域内中心像素的灰度值, i_p 则为其邻域像素的灰度值, $\text{sgn}(x)$ 是符号函数。

· 实验步骤

- Step1: 导入数据, 将图片转化为灰度形式;
- Step2: 定义两个函数: `calculate_lbp_pixel` 和 `local_binary_pattern`。`calculate_lbp_pixel` 函数用于计算某个像素的 LBP 值, `local_binary_pattern` 函数用于生成整幅图像的 LBP 图像。
- Step3: 对图片使用上述定义函数进行特征提取, 将结果转化为直方图, 和 LBP 提取后的图像同时输出;

2.4 实验结论与分析



从图像直接观察 : forest 图片之间较难分辨差别, cloud 图片之间差异较大。

从直方图观察 : forest 仅在两侧有较高峰值, 中间绝大部分的频数均较低; cloud 的峰分布较为均匀, 不同值的峰在整个 x 轴的不同位置均有出现。

经分析, 这种分布特征可能代表的结论如下:

Forest 图像的纹理主要集中在局部细节上 (树木、叶子), 而整体分布较为均匀, 因此导致了 LBP 算子极小和极大的概率增加;

而与 forest 相比, Cloud 图像可能具有更加丰富和变化的纹理特征, 这些纹理可能在不同区域呈现出较大差异, LBP 算子也因此有更多机会取到更多不同的值。

2.5 实验代码

```
1 import os
2 import numpy as np
3 from PIL import Image
4 import matplotlib.pyplot as plt
5
6 def calculate_lbp_pixel(image, i, j):
7     center = image[i, j]
8     code = 0
9     if image[i-1, j-1] >= center:
10         code += 128
11     if image[i-1, j] >= center:
12         code += 64
13     if image[i-1, j+1] >= center:
14         code += 32
15     if image[i, j+1] >= center:
16         code += 16
17     if image[i+1, j+1] >= center:
18         code += 8
19     if image[i+1, j] >= center:
20         code += 4
21     if image[i+1, j-1] >= center:
22         code += 2
23     if image[i, j-1] >= center:
24         code += 1
25     return code
26
27 def local_binary_pattern(image, n_points, radius):
28     height, width = image.shape
29     lbp_image = np.zeros(image.shape, dtype=np.uint8)
30
31     for i in range(radius, height-radius):
32         for j in range(radius, width-radius):
33             code = calculate_lbp_pixel(image, i, j)
34             lbp_image[i, j] = code
```

```
35
36     return lbp_image
37
38 def process_images_in_folder(folder_name, label):
39     fig, axes = plt.subplots(2, 10, figsize=(20, 5))
40     axes = axes.ravel()
41
42     for i in range(10):
43         img_name = f"{i+1}.png"
44         img_path = os.path.join(folder_name, img_name)
45         img = Image.open(img_path).convert('L')
46
47         height, width = img.size
48         image_array = np.array(img)
49
50         n_points, radius = 8, 3
51         lbp = local_binary_pattern(image_array, n_points, radius)
52
53         n_bins = int(2 ** n_points)
54         hist, bin_edges = np.histogram(lbp, bins=n_bins, range=(0,
55 n_bins), density=True)
56
57         axes[i].imshow(lbp, cmap='gray')
58         axes[i].set_title('LBP Image {}'.format(i+1))
59
60         axes[i+10].bar(np.arange(n_bins), hist, width=0.8, align='center',
61 color='g', edgecolor='k')
62         axes[i+10].set_title('LBP Histogram {}'.format(i+1))
63         axes[i+10].set_ylim([0, hist.max() * 1.1])
64         axes[i+10].set_ylabel('Percentage')
65
66     for ax in axes:
67         ax.axis('off')
68
69     fig.suptitle(label)
70     plt.show()
71
72 process_images_in_folder('cloud', 'Cloud Images')
73 process_images_in_folder('forest', 'Forest Images')
```

第三章 第三次上机实验 (朴素 Bayes 预测)

3.1 实验要求

- 给定一定时间内的犯罪数据集，使用朴素贝叶斯方法进行犯罪类型预测
- 使用 Python 编程实现

3.2 数据分析与处理

· 数据分析

训练集与测试集格式相同，包含了“时间”（包含日期与精确到秒的具体时间），“犯罪类型”，“描述”，“星期”，“案发街区”等信息。其中，“描述”是无需关注的变量；“犯罪类型”是我们的分类标签响应变量，一共有 6 种类型；“时间”“星期”以及“案发街区”是自变量，即实验的目标是：通过朴素贝叶斯算法，使用这三个自变量对“犯罪类型”这一分类标签型的响应变量做预测分析。

· 数据处理

对于“时间”，“星期”和“案发街区”这三个文本与数字混合的自变量，为了后续训练预测过程的方便调用，使用 one-hot 编码方法进行处理，处理后每一个变量都对应着一个唯一的二进制向量。

其中，“时间”被编码为 23 维的 0-1 向量，“星期”被编码为 7 维的 0-1 向量（共 7 个取值），“案发街区”被编码为 10 维的 0-1 向量（共 10 个街区）。

3.3 实验步骤与原理

· 实验原理

本实验的原理是朴素贝叶斯。首先，由 Bayes 公式：

$$P(\theta|X) = \frac{P(X|\theta) \times P(\theta)}{P(X)} \quad (3.1)$$

同时，我们假设目标值给定时，属性值互相独立：

$$P(a_1, \dots, a_n|v_j) = \prod_i P(a_i|v_j) \quad (3.2)$$

综上，在给定一组待分类的属性值时，朴素 Bayes 分类准则如下：

$$v_{NB} = \arg \max_{x \in S} P(v_j) \prod_i P(a_i|v_j) \quad (3.3)$$

· 实验步骤

- Step 1: 读取数据, 使用 one-hot 编码方法将全部自变量转化为二进制 0-1 向量;
- Step 2: 根据上述计算式, 定义先验概率与条件概率计算公式;
- Step 3: 使用上面定义的概率计算函数, 根据朴素 Bayes 原理进行预测;
- Step 4: 输出预测准确率, 以及可视化的混淆矩阵, 分析实验结果。

3.4 实验结论与分析

预测的准确率为 90.72948332689277%., 超过了 90%, 说明预测相对比较准确。

3.5 实验代码

```
1 import os
2 import pandas as pd
3 from sklearn import *
4 import numpy as np
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7
8 train_path = 'train.csv'
9 test_path = 'test.csv'
10 train_df = pd.read_csv(train_path)
11 test_df = pd.read_csv(test_path)
12
13 # 数据预处理
14 encoder = preprocessing.LabelEncoder()
15 crime_type_encode = encoder.fit_transform(train_df['Category'])
16 crime_type_encode_t = encoder.fit_transform(test_df['Category'])
17 hour = pd.to_datetime(train_df['Dates']).dt.hour
18 hour = pd.get_dummies(hour)
19 day = pd.get_dummies(train_df['DayOfWeek'])
20 hour_t = pd.to_datetime(test_df['Dates']).dt.hour
21 hour_t = pd.get_dummies(hour_t)
22 day_t = pd.get_dummies(test_df['DayOfWeek'])
23 police_district = pd.get_dummies(train_df['PdDistrict'])
24 police_district_t = pd.get_dummies(test_df['PdDistrict'])
25 train_set = pd.concat([hour, day, police_district], axis = 1)
26 train_set['Crime type'] = crime_type_encode
27 test_set = pd.concat([hour_t, day_t, police_district_t], axis = 1)
```

```
28 test_set['Crime type'] = crime_type_encode_t
29
30 # 提取自变量和因变量
31 X_train = train_set.iloc[:, :40].values
32 y_train = train_set.iloc[:, 40].values
33 X_test = test_set.iloc[:, :40].values
34 y_test = test_set.iloc[:, 40].values
35
36 # 计算先验概率P(class)
37 def calculate_prior(labels, class_value):
38     return np.sum(labels == class_value) / len(labels)
39
40 # 计算某个特征下, 属于某个类别的条件概率P(feature|class)
41 def calculate_conditional_probability(feature, feature_value, labels,
42     class_value, epsilon=1e-5):
43     numerator = np.sum((labels == class_value) & (feature ==
44         feature_value))
45     denominator = np.sum(labels == class_value)
46     return (numerator + epsilon) / (denominator + epsilon*len(np.unique(
47         feature)))
48
49 # 使用贝叶斯定理进行分类
50 def predict(features, labels, new_feature):
51     unique_classes = np.unique(labels)
52     best_class = None
53     best_prob = -1
54
55     for class_value in unique_classes:
56         prior = calculate_prior(labels, class_value)
57         conditional_prob = 1
58         for i, feature_value in enumerate(new_feature):
59             conditional_prob *= calculate_conditional_probability(
60                 features[:,i], feature_value, labels, class_value)
61         probability = prior * conditional_prob
62         if probability > best_prob:
63             best_prob = probability
64             best_class = class_value
65     return best_class
66
67 # 预测测试集
68 predictions = [predict(X_train, y_train, x) for x in X_test]
69 # 计算准确率
70 accuracy = accuracy_score(y_test, predictions)
71 print("准确率:", accuracy)
```

第四章 第四次上机实验（决策树预测）

4.1 实验要求

- 给定一定时间内的犯罪数据集，使用决策树方法进行犯罪类型预测
- 使用 Python 编程实现

4.2 数据分析与处理

· 数据分析

训练集与测试集格式相同，包含了“时间”（包含日期与精确到秒的具体时间），“犯罪类型”，“描述”，“星期”，“案发街区”等信息。其中，“描述”是无需关注的变量；“犯罪类型”是我们的分类标签响应变量，一共有 6 种类型；“时间”“星期”以及“案发街区”是自变量，即实验的目标是：通过朴素贝叶斯算法，使用这三个自变量对“犯罪类型”这一分类标签型的响应变量做预测分析。

· 数据处理

对于“时间”，“星期”和“案发街区”这三个文本与数字混合的自变量，为了后续训练预测过程的方便调用，使用 one-hot 编码方法进行处理，处理后每一个变量都对应着一个唯一的二进制向量。

其中，“时间”被编码为 23 维的 0-1 向量，“星期”被编码为 7 维的 0-1 向量（共 7 个取值），“案发街区”被编码为 10 维的 0-1 向量（共 10 个街区）。

4.3 实验步骤与原理

· 实验原理

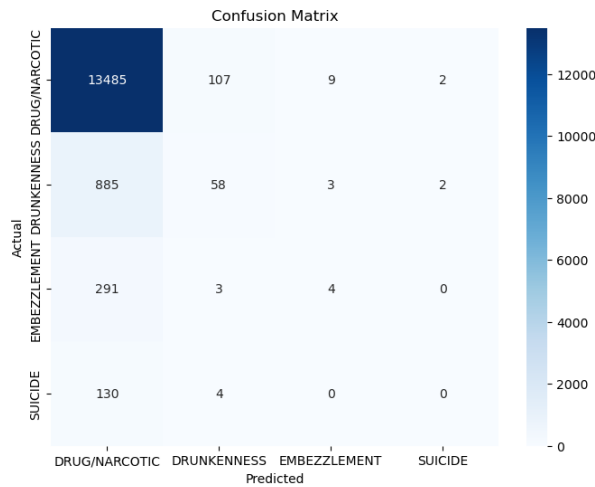
决策树的一般原理：从根节点出发，对实例的每一个特征计算不纯度指标，根据判断结果，将实例分配到其子节点中，直至将实例分配到叶子结点中。随着划分过程不断进行，决策树的分支结点所包含的样本尽可能属于同一类别，即结点的“纯度” (purity) 越来越高。

本实验使用 CART 决策树进行预测分析。

· 实验步骤

- Step 1: 读取数据，使用 one-hot 编码方法将全部自变量转化为二进制 0-1 向量；
- Step 2: 调取 sklearn.tree 中 DecisionTreeClassifier 包，使用在训练集上实现决策树的构建，并在测试集上预测；
- Step 3: 输出预测准确率，绘制可视化混淆矩阵，进行结果分析。

4.4 实验结论与分析



预测准确率为 **90.41580457852233%**。

从混淆矩阵可以发现，即使准确率较高，但仍有较大的第二类错误，非对角元仍有相对较大的误判值。观察数据分布，发现标签为 0 的样本数量很大，推测可能是由于样本分布不均衡导致的模型失真，因此有一定的误判率，可以考虑使用剪枝或重采样等处理方法进一步优化模型及预测结果。

4.5 实验代码

```
1 import os
2 import pandas as pd
3 from sklearn import *
4 import numpy as np
5
6 train_path = 'train.csv'
7 test_path = 'test.csv'
8 train_df = pd.read_csv(train_path)
9 test_df = pd.read_csv(test_path)
10
11 # 将Category进行整数编码
12 encoder = preprocessing.LabelEncoder()
13 crime_type_encode = encoder.fit_transform(train_df['Category'])
14 crime_type_encode_t = encoder.fit_transform(test_df['Category'])
15
16 # 将时间进行one-hot编码
17 hour = pd.to_datetime(train_df['Dates']).dt.hour
18 hour = pd.get_dummies(hour)
```



```
19 day = pd.get_dummies(train_df['DayOfWeek'])
20 hour_t = pd.to_datetime(test_df['Dates']).dt.hour
21 hour_t = pd.get_dummies(hour_t)
22 day_t = pd.get_dummies(test_df['DayOfWeek'])
23
24 # 将所属警区进行one-hot编码
25 police_district = pd.get_dummies(train_df['PdDistrict'])
26 police_district_t = pd.get_dummies(test_df['PdDistrict'])
27 train_set = pd.concat([hour, day, police_district], axis = 1)
28 train_set['Crime type'] = crime_type_encode
29 test_set = pd.concat([hour_t, day_t, police_district_t], axis = 1)
30 test_set['Crime type'] = crime_type_encode_t
31
32 # 建立决策树模型
33 from sklearn.metrics import accuracy_score
34 clf = DecisionTreeClassifier()
35
36 # 提取特征和标签列
37 X_train = train_set.drop('Crime type', axis=1) # 特征列
38 y_train = train_set['Crime type'] # 标签列
39 X_test = test_set.drop('Crime type', axis=1) # 测试集特征列
40 y_test = test_set['Crime type'] # 测试集标签列
41
42 # 训练模型及预测
43 clf.fit(X_train, y_train)
44 predictions = clf.predict(X_test)
45
46 # 计算准确率
47 accuracy = accuracy_score(y_test, predictions)
48 print("Decision Tree Classifier accuracy:", accuracy)
49
50 import matplotlib.pyplot as plt
51 from sklearn.metrics import confusion_matrix
52 import seaborn as sns
53
54 # 计算混淆矩阵
55 conf_matrix = confusion_matrix(y_test, predictions)
56 plt.figure(figsize=(8, 6))
57 sns.heatmap(conf_matrix, annot=True, cmap="Blues", fmt="d", xticklabels=
    encoder.classes_, yticklabels=encoder.classes_)
58 plt.xlabel('Predicted')
59 plt.ylabel('Actual')
60 plt.title('Confusion Matrix')
61 plt.show()
```

第五章 第五次上机实验 (BP 神经网络)

5.1 实验要求

- 给定信用卡欺诈数据集，判断是否存在欺诈
- 请使用 BP 神经网络或基于 delta 准则的感知机实现

注：本实验使用 BP 神经网络完成

5.2 数据分析与处理

在 284807 笔交易中，共有 492 笔欺诈交易，仅占总数据量的 0.172%。训练集共有 10300 笔交易数据，其中欺诈交易 300 笔。测试集共有 4507 笔交易数据，其中欺诈交易 192 笔。

自变量共 30 维，其中 $V_1, V_2 \dots V_{28}$ 是 PCA 得到的主成分（为了保护原始数据隐私安全），没有经过 PCA 变换的特征是时间和金额。“Class” 是响应变量，是“遭遇欺诈”的示性函数。

在导入数据后，为了保证神经网络的训练稳定和收敛性，防止梯度爆炸等问题的出现，需要对自变量进行标准化。

5.3 实验步骤与原理

· 实验原理

BP 神经网络是一个三层的前馈网络。为了适应于梯度下降算法，我们在每一层之间添加激活函数。首先初始化模型的参数，然后使用前向传播和反向传播的交替步骤来更新模型参数，根据反向传播计算得到的梯度，结合梯度下降算法进行参数更新。对于不平衡分类问题，我们使用精度召回曲线下面积（AUC）作为准确率的衡量指标。

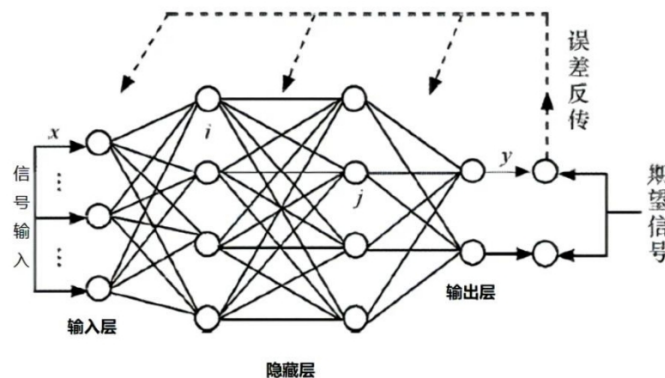


图 5.1: Mechanism of BP

· 实验步骤

- Step 1: 清洗数据集, 将其使用 StandardScaler 进行标准化处理;
- Step 2: 创建一个神经网络模型。在 tensorflow 中使用 tf.keras.Sequential 来构建一个序贯模型。在模型中添加多层全连接的神经网络层, 选定合适的激活函数;
- Step 3: 选择 'binary_crossentropy' 作为损失函数, 'adam' 作为优化器, 添加评估指标;
- Step 4: 训练模型, 分别使得 epoch=100/300/500, 并计算 AUPRC 的值, 绘制可视化图表。

5.4 实验结论与分析

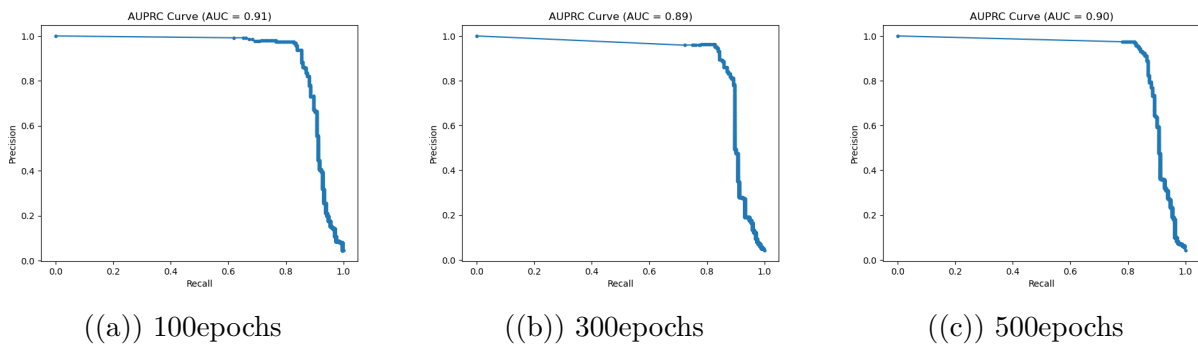


图 5.2: 不同 epochs 下 AUPRC 曲线

该神经网络的预测准确率为 **99.11249167960949%**, AUPRC 在不同 epoch 下均在 0.9 附近波动。

本实验仅选取 100/300/500epochs 三种情况运行以观察规律。在 300epoch 下, AUPRC=0.89, 略低于 100epoch 和 500epoch 时的 AUPRC, 推测可能是因为过拟合或学习率不当导致的, 可以考虑改变 loss 与优化器实现预测效果的进一步提高。

5.5 实验代码

```
1 import os
2 import pandas as pd
3 import tensorflow as tf
4 from sklearn.metrics import precision_recall_curve, auc
5 import matplotlib.pyplot as plt
6 from sklearn.preprocessing import StandardScaler
```

```
7
8 #读取数据
9 train =pd.read_csv('train_5.csv')
10 test =pd.read_csv('test_5.csv')
11 x_train=train.iloc[:,1:31]
12 x_test=test.iloc[:,1:31]
13 y_train=train.iloc[:,31]
14 y_test=test.iloc[:,31]
15
16 #数据标准化
17 from sklearn.preprocessing import StandardScaler
18 scaler = StandardScaler()
19 x_train = scaler.fit_transform(x_train)
20 x_test = scaler.fit_transform(x_test)
21
22 # 构建神经网络模型
23 model = tf.keras.Sequential([
24     tf.keras.layers.Dense(64, activation='relu', input_shape=(30,)),
25     tf.keras.layers.Dense(32, activation='relu'),
26     tf.keras.layers.Dense(1, activation='sigmoid')
27 ])
28
29 # 编译模型
30 model.compile(optimizer='adam',
31               loss='binary_crossentropy',
32               metrics=['AUC', 'Precision', 'Recall'])
33
34 # 训练模型
35 model.fit(x_train_scaled, y_train, epochs=500, batch_size=32,
36           validation_data=(x_test_scaled, y_test))
37
38 # 预测
39 y_pred = model.predict(x_test_scaled)
40
41 precision, recall, _ = precision_recall_curve(y_test, y_pred)
42 auprc = auc(recall, precision)
43
44 # 绘制AUPRC曲线
45 plt.plot(recall, precision, marker='.')
46 plt.xlabel('Recall')
47 plt.ylabel('Precision')
48 plt.title('AUPRC Curve (AUC = %0.2f)' % auprc)
49 plt.show()
```